Jose Naranjo

Infrastructure / DevOps Engineer

Alajuela, Costa Rica (UTC−6, no DST) · **online from 06:00 CR — 4h overlap with a 09:00–18:00 CEST day**

jmnaranjo09@gmail.com · English C1, Spanish native

josenaranjo.website · linkedin.com/in/Jose-Naranjo-3b1a13165 · github.com/jnaranj09

Infrastructure engineer, 4 years on self-managed production Kubernetes and the CI/CD around it, 8 years in infrastructure overall. Two things I do most. First, **per-PR / ephemeral environments**, so every feature gets a full stack of its own. Second, **cutting operational toil with AI tooling**: an 11-skill agent-consumed operations library that runs real production deploys, in daily use. I own the rollback path, carry the pager, and turn incidents into permanent checks instead of repeat tickets.

EXPERIENCE

DevOps Engineer — Clicklease · June 2022 – Present · Remote

Developer environments and internal tooling

- Built **per-namespace feature environments** that come up in **5 minutes** — a full stack per ticket, ~**88 workloads** (52 microservices, 21 databases, 5 frontends), end-to-end tests included. **32 run side by side today**; the only slow dependency is a Salesforce sandbox (~40 min).
- Authored an **11-skill operations library consumed by AI agents** that performs real production deploys, merges, tag promotion and environment config changes. In daily use.
- Built **Salesforce developer-sandbox automation** — 47 commits — so developers get their own sandbox without a manual request, plus the **email-deliverability CLI** the scripts call (repo). **Adopted by other people at work, including our Salesforce architect.**
- Co-own a **shared GitLab CI template library of 70+ pipelines** that every service, database and frontend repo includes instead of carrying its own — **425 commits** of mine.

Keycloak config as code

- Replaced a **destructive export → override → reimport cycle that required downtime** with online, non-destructive reconciliation of all **5 Keycloak realms**. **ADR moved from Proposed to Accepted**; normalization took the config from **14,797 raw JSON lines to 4,552 lines of jsonnet**.
- The pilot caught four config types the old tool was **silently deleting on every apply** (console-added scopes, auth flows, identity providers and groups), and a creation-time default drift that would have hit **48 clients** in any freshly created environment.
- Built the **self-service diff tool** on top: a coworker changes something in the admin console and gets back the **exact config file and line** to edit in git. It also runs **on a schedule to flag unexpected drift**. A self-test CI job sweeps all 5 realms against a known-in-sync environment — exception allowlist driven to **zero**.
- Configured **HAProxy with the Keycloak RS256 certificate** so **JWTs are validated at the edge**: public requests carrying an invalid token are rejected before they reach any service.

Release engineering

- Moved a **large .NET portal off manual Visual Studio builds**: it used to be built by hand in ~**30 minutes** over several steps, then the artifact was copied between Windows servers. GitLab CI now builds it with **MSBuild.exe in ~10 minutes** and stores the artifact centrally; a second pipeline deploys it. Artifacts are tagged and restorable, so we can **roll back a bad production build** and **deploy to several environments at once**.
- Formalized a **4-step post-deploy image verification procedure** and wired it into the deploy tooling: the tag has the expected changes, the pipeline SHA matches the tag's SHA, the running pod's digest matches the built digest, and no stale pods remain. **Success notifications are now gated on all four.** Before that they fired on pipeline success alone, with no check that the running pod reflected the new build.

Incidents and root cause

- Root-caused a **cluster-wide control-plane failure**: a Kubernetes upgrade left the control-plane pods un-restarted, so `kube-controller-manager` had lost authentication to the API server and **stopped reconciling Deployments across the whole cluster**. One stuck rollout was the symptom everyone watched; the upgrade was the cause. Restarting the control-plane pods restored reconciliation.
- Found a **Spring Boot endpoint whose response never matched the shared-library DTO**: the field names differed, so every consuming service had been silently deserializing nulls. Nothing broke, because that flow never used those values. The **Java 25 / Spring Boot 4 / Jackson 3** upgrade exposed it — Jackson stopped defaulting missing fields to null and the endpoint began throwing.
- Chase the **pattern behind a single failure**: load-balancer security configuration drifting between environments because each one carried its own copy instead of a shared template; tests failing in feature environments because database tables were missing unique constraints.

Infrastructure-side security

- Found a tool in our Microsoft 365 tenant holding **application-level Graph permissions far beyond its job**: with only a user's directory object id it could read **any employee's private 1:1 and group chats**, no consent needed. Reproduced it deliberately and reported it to IT.
- Added the **certificate-expiry monitoring that was missing**: expired certificates had been taking services down, so expiry now goes into Prometheus and alerts fire **before** a certificate dies instead of after the outage.

Cost, build-vs-buy, and observability

- Evaluated building a custom service against adopting an existing one, chose **self-hosted n8n**, and shipped a production document-ingestion workflow that **replaced a paid third-party product, removing the recurring license entirely**.
- Built and run the **production OpenSearch cluster**: **~6 billion documents / 1.6 TB**, ingesting ~**20 GB/day** of logs and OpenTelemetry traces from all three environments through **Data Prepper** pipelines, with full TLS, internal auth and Dashboards, and about a year of retention. Cut stored log size by **filtering unused fields out in Data Prepper before indexing**.

Platform and cloud

- Operate **3 self-managed Kubernetes clusters** (dev, stage, production) — **51 nodes, ~3,300 pods** — under real traffic: HPA tuning, resource requests and limits, namespace-scoped RBAC. Carry the pager, run incidents, and write the follow-up.
- Administer the **Linux servers underneath**: provisioning and config management with **Foreman and Puppet**, monitoring with **Zabbix**, and **LVM** volume management.
- **Migrated EC2 workloads to Kubernetes and Docker**, and built the infrastructure monitoring on **Grafana, Prometheus and OpenTelemetry**.
- **Authored and applied Terraform** provisioning AWS **PrivateLink** (VPC endpoint service, internal NLB, target group, listener, security group) to expose an internal GitLab instance to Snowflake. Also integrated **S3, Transit Gateway and Site-to-Site VPN**.

Technical Support Engineer — VMware · September 2020 – June 2022

- Resolved **business-critical customer outages** on VMware SRM and vSAN. Debugged across Cloud Foundation and vCenter Server, ran best-practice consultations, and trained new team members.

IT Administrator — NETWAY · November 2017 – September 2020

- Built wired and wireless networks and virtualization infrastructure, deployed endpoint security, and led customer-facing implementation meetings.

PERSONAL PROJECTS

- **claude-watcher** (repo). Electron app that watches several Claude Code sessions at once and flags which one is working, which needs you, and which is idle. **800+ tests**, e2e green. Built it for myself, then **a coworker started using it too**.
- **Self-hosted personal infrastructure** on `josenaranjo.website` — my own Linux host behind **Cloudflare DNS and Cloudflare Tunnel**, running a self-hosted **n8n** instance and serving this CV. No cloud platform in the path.
- **The demo** on `demo.josenaranjo.website` — same host: a **k3s** cluster **ArgoCD** deploys straight from GitHub, an **HAProxy** edge, and a **local qwen2.5 model**. A visitor can ask its assistant for the light theme; that **pushes a real commit** and ArgoCD rolls it out.

SKILLS

Orchestration Kubernetes (self-managed, production, 3 clusters) · Helm · HPA and resource tuning · namespace RBAC · ephemeral / per-PR environments

CI/CD GitLab CI (production: pipeline templates, tag promotion, digest verification) · GitHub Actions (personal projects) · self-hosted runners

IaC & config management Terraform (authored and applied) · Puppet · Foreman · jsonnet · AWS (S3, IAM, EC2, Transit Gateway, Site-to-Site VPN, PrivateLink) · Cloudflare DNS and Tunnel

Observability OpenTelemetry · OpenSearch · Data Prepper · Grafana · Prometheus · Zabbix · Loki · Incident.io · PagerDuty

Data & messaging PostgreSQL · RabbitMQ

Identity Keycloak (config as code, realm reconciliation, OIDC clients)

Platform & languages Linux (LVM, systemd) · HAProxy · Docker · Git · Bash · Python · JavaScript / TypeScript · Java / Spring Boot (troubleshooting)

AI tooling Claude Code skills, rules and durable memory running real production operations · self-hosted n8n

CERTIFICATIONS AND EDUCATION

VMware Certified Implementation Expert (VCIX) — VMware Academy, 2022 · **VMware vSAN Specialist** — VMware, 2021 · **Cisco CCNA** — Cisco Networking Academy, 2020 · **Windows Server Administration** — Cenfotec University, 2022 · **High School Bachelor with IT Support Certification** — 2017